

Boolean Arithmetic & the ALU

Lab 2 — Nand2Tetris Project 2

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 21, 2026

Aim. *To design half-adder, full-adder and multi-bit adder circuits, and to build the 16-bit Arithmetic-Logic Unit (ALU) that lies at the heart of the Hack CPU, entirely as combinational HDL circuits.*

2.1 Learning Outcomes

On completing this lab, a student will be able to:

- Explain binary addition and two’s-complement representation of negative numbers.
- Build a ripple-carry adder from half-adders and full-adders.
- Interpret the ALU’s six control bits (**zx**, **nx**, **zy**, **ny**, **f**, **no**) and derive the function they select.
- Design and test a 16-bit ALU that performs 18 distinct arithmetic/logical operations from one circuit.

2.2 Pre-Lab Reading

Pre-Lab Reading. This lab’s prerequisite theory was already introduced in **Week 2** — review that lecture’s treatment of two’s-complement addition/subtraction, the half-adder (HA) and full-adder (FA) Boolean equations $S = A \oplus B$, $C = A \cdot B$ (HA) and $S = A \oplus B \oplus C_{in}$, $C_{out} = A \cdot B + C_{in}(A \oplus B)$ (FA), the 4-bit ripple-carry adder, and the “subtraction is free” idea (compute $A - B$ as $A + \overline{B} + 1$ via a MUX plus a forced carry-in) — the last of these directly foreshadows this lab’s **f/no** control bits. Page-verified source for that lecture’s citations: Patterson & Hennessy, *Computer Organization and Design, ARM Edition*, §2.4 *Signed and Unsigned Numbers* (p. 75), §3.2 *Addition and Subtraction* (p. 188).

What this lab adds that Week 2 does not cover: the Nand2Tetris ALU’s specific 6-control-bit API and 18-operation selection table (Section 2.4 below). For general background on multifunction-ALU construction from an adder, see Patterson & Hennessy Appendix A — §A.5 *Constructing a Basic Arithmetic Logic Unit* (p. A-26, Fig. A.5.6) and §A.6 *Faster Addition: Carry Lookahead* (p. A-37); this is general treatment of ALU design, not a page-precise description of Nand2Tetris’s own **zx/nx/zy/ny/f/no** scheme. Hamacher Ch. 9 and Mano Ch. 1 (both cited for Week 2) are not page-indexed here — chapter-level citation only, per the textbook-grounding rule.

Also review the Lab 1 gate library (**Not**, **And**, **Or**, **Xor**, **Mux**, **Not16**, **And16**, **Or8Way**) — every chip in this lab reuses it.

2.3 Toolchain Setup

1. Install a Java Runtime Environment (JRE), version 8 or later, if not already present.
2. Download the Nand2Tetris software suite (free, nand2tetris.org) — this lab uses the **Hardware Simulator** only; the CPU Emulator and Assembler are needed from Lab 4 onward.
3. Extract the suite and locate the `projects/02/` folder — it contains one subfolder per chip, pre-populated with a `ChipName.hdl` stub (to be completed) and the supplied `ChipName.tst` / `ChipName.cmp` test/comparison scripts. These scripts are standard Nand2Tetris project files and are *not* redistributed here — obtain them from the official suite.
4. Open the Hardware Simulator, load a chip's `.tst` script (FILE → LOAD SCRIPT), and run it (SIMULATION → RUN). A correct chip prints `End of script --- comparison ended successfully` in the script's output pane; a wrong one reports the first mismatching row.
5. For a full walkthrough of the Hardware Simulator's interface, see the official *Hardware Simulator Tutorial* (Nand2Tetris, nand2tetris.org).

2.4 Tasks

Implement the following 5 chips **in order** — each depends on the ones before it, and all reuse Lab 1's gate library.

1. **HalfAdder(a,b) → (sum,carry)** — $sum = a \oplus b$, $carry = a \cdot b$. Built from **Xor** and **And**.
2. **FullAdder(a,b,c) → (sum,carry)** — adds three bits (a , b , and carry-in c). Hint: two **HalfAdders** plus an **Or** on the two carry outputs.
3. **Add16(a[16],b[16]) → out[16]** — a 16-bit adder: a ripple-carry chain of 16 full adders (bit 0 uses a **HalfAdder**, since there is no carry-in at the least-significant bit). The carry-out of bit 15 is discarded (two's-complement overflow is not flagged by this chip).
4. **Inc16(in[16]) → out[16]** — $out = in + 1$. Hint: this is exactly **Add16** with the constant 1 as the second operand; a ripple chain of **HalfAdders** (bit 0 adds the constant **true**, every later bit adds the previous bit's carry) is the standard implementation.
5. **ALU(x[16],y[16],zx,nx,zy,ny,f,no) → (out[16],zr,ng)** — the 16-bit Arithmetic-Logic Unit. Six control bits select among 18 operations:
 - (a) If **zx**: zero out x .
 - (b) If **nx**: bitwise-negate x .
 - (c) If **zy**: zero out y .
 - (d) If **ny**: bitwise-negate y .
 - (e) If **f**: $out = x + y$ (via **Add16**); else $out = x \& y$ (via **And16**).
 - (f) If **no**: bitwise-negate out .
 - (g) $zr = 1$ iff $out = 0$ (all 16 bits zero); $ng = 1$ iff out 's sign bit (bit 15) is set.

zx	nx	zy	ny	f	no	out
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	$\bar{x}$
1	1	0	0	0	1	$\bar{y}$
0	0	1	1	1	1	$-x$
1	1	0	0	1	1	$-y$
0	1	1	1	1	1	$x + 1$
1	1	0	1	1	1	$y + 1$
0	0	1	1	1	0	$x - 1$
1	1	0	0	1	0	$y - 1$
0	0	0	0	1	0	$x + y$
0	1	0	0	1	1	$x - y$
0	0	0	1	1	1	$y - x$
0	0	0	0	0	0	$x \& y$
0	1	0	1	0	1	$x y$

Table 1: The 18 ALU operations selected by the six control bits (the ALU’s official Nand2Tetris specification).

Table 1 shows how the six control bits select all 18 required operations — this is the ALU’s complete functional specification.

Load each chip’s supplied `.tst` script in the Hardware Simulator; for ALU run *both* `ALU.tst` and the stricter `ALU-nostat.tst` (which disallows shortcuts such as detecting the control-bit pattern and returning a hard-coded constant).

2.5 Figures

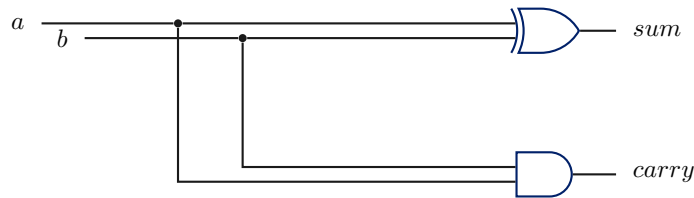


Figure 1: HalfAdder at gate level: $sum = a \oplus b$ (Xor), $carry = a \cdot b$ (And). Exactly two gates — this is the module FullAdder reuses twice.

2.6 Truth Tables & Boolean Expressions (step by step)

HalfAdder (Figure 1) — one bit of addition with no carry-in:

$$sum = a \oplus b, \quad carry = a \cdot b$$

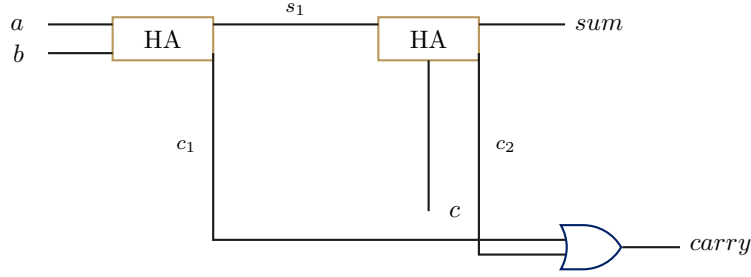


Figure 2: **FullAdder** = two **HalfAdders** plus one **Or**: $s_1 = a \oplus b$, $c_1 = a \cdot b$; then $sum = s_1 \oplus c$, $c_2 = s_1 \cdot c$; finally $carry = c_1 + c_2$. Each HA block expands to Figure 1.

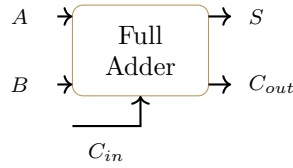


Figure 3: The full adder as a block: A , B and C_{in} in, S and C_{out} out — **FullAdder**’s I/O contract ($a,b,c \rightarrow sum, carry$ in the HDL).

FullAdder — two half adders plus an **Or** (Figure 2). Step by step:

$$\begin{aligned}
 \text{(HA1)} \quad & s_1 = a \oplus b, \quad c_1 = a \cdot b \\
 \text{(HA2)} \quad & sum = s_1 \oplus c = a \oplus b \oplus c \\
 & c_2 = s_1 \cdot c = (a \oplus b) \cdot c \\
 \text{(Or)} \quad & carry = c_1 + c_2 = a \cdot b + (a \oplus b) \cdot c
 \end{aligned}$$

Add16 / **Inc16** are 16 parallel copies of the adder above: $out = a + b$ (carry-out of bit 15 discarded), and $out = in + 1$ (the constant-1 ripple chain). Two’s-complement subtraction is “free”: $x - y = x + \bar{y} + 1$, which the ALU realises via the **nx/ny** (no) negate bits and a forced carry-in — see Table 1.

2.7 Observation / Journal Table

Record, for each chip, the gates used in your composition and whether it passed its supplied test script.

Journal instruction (separate from the table above): tabulate, for at least *six* control-bit combinations of your choice (drawn from or beyond Table 1), the operation each performs — e.g. $0101010 \rightarrow out = x - y$. This is the syllabus’s own Deliverable 2.

2.8 Viva-Voce Questions

1. Explain binary addition and how two’s-complement handles negative numbers without a separate subtractor circuit.
2. Why does **Inc16** not need a full **Add16** internally? What does it reuse instead?
3. Trace ALU by hand for $zx=0, nx=0, zy=1, ny=1, f=0, no=0$ (x AND y ’s zeroed value). What is out ?

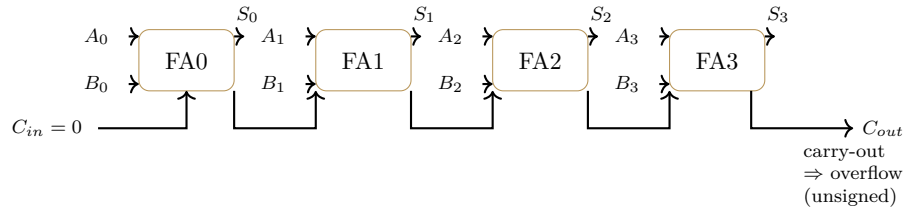


Figure 4: The ripple-carry adder: full adders chained by their carry signals. **Add16** is this same pattern extended to 16 bits, with bit 0 using a **HalfAdder** (no carry-in) instead of an **FA0**.

inputs		outputs	
a	b	$sum = a \oplus b$	$carry = a \cdot b$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 2: **HalfAdder** truth table.

4. Why is **Add16**'s carry-out from bit 15 simply discarded rather than reported as an overflow flag?
5. What would happen to **zr** if **Or8Way** were applied to only 8 of the 16 output bits instead of all 16?

2.9 Rubric

Reproduced verbatim from `syllabi/CS401_Lab_Syllabus_Nand2Tetris.pdf`.

inputs			outputs	
<i>a</i>	<i>b</i>	<i>c_{in}</i>	<i>sum</i>	<i>carry</i>
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 3: `FullAdder` truth table (*sum* is the XOR of the three inputs; *carry* is set whenever two or more inputs are 1).

Chip	Gates used (internal composition)	Pass / Fail
<code>HalfAdder</code>		
<code>FullAdder</code>		
<code>Add16</code>		
<code>Inc16</code>		
<code>ALU (ALU.tst)</code>		
<code>ALU (ALU-nostat.tst)</code>		

Table 4: Lab 2 observation table — one row per chip (ALU tested against both scripts).

Criterion	Marks
ALU passes <code>ALU-nostat.tst</code> (strict, no shortcuts)	5
Add16 / Inc16 / adders correctly reused inside the ALU	2
Control-bit truth table complete and correct	2
Viva: can trace out for a given control-bit combination by hand	1
Total	10